Introduction to Software Engineering

Soha Makady

s.makady@fci-cu.edu.eg

About Us

- Dr. Manar El-Kady: m.elkady@fci-cu.edu.eg
  - Office hours: Right after the lecture, or by appointment (through email)
  - Office: 2nd floor in the New Student Building, in the main campus, right beside the electronics lab.
- Dr. Soha Makady: s.makady@fci-cu.edu.eg
  - Office hours: Right after the lecture, or by appointment (through email)
  - Office: 3rd floor in the Main Building.

Google Classroom

- https://classroom.google.com/c/NjY0MDgzODkyMDQx?cjc=cbuihdm
- Class code: cbuihdm

Outline

- **Is Software such a Challenging Thing?**
- What is Software Engineering?
- Software Project Failures
- Software Development Myths
- Steps to a Successful Software
- A Case Study
- Course Organization

Is Software Such a Challenging Thing?

What is Software?

Software is:

- (1) **instructions (computer programs)** that when executed provide desired features, function, and performance;
- **(2) data structures and algorithms** that enable the programs to adequately manipulate information, and
- (3) **documentation** in both hard copy and virtual forms that describes the operation and use of the programs

Is Software Such a Challenging Thing?

Any other definition for Software?

- "Software is a place where **dreams are planted** and **nightmares harvested**,
- an abstract, mystical swamp where terrible **demons compete** with **magical panaceas**,
- a world of **werewolves** and **silver bullets**." **Brad J. Cox**
- **What makes software such a challenge for software engineers?**

Challenge 1: Software Application Domains

- The presence of several broad categories of computer software present continuing challenge for software engineers.
- Such categories include:
  - **System software** -- a collection of programs written to service other programs (e.g., compilers, editors, file managers)
  - **Application software** -- stand-alone programs that solve a specific business need (e.g., e-com, tanseeq)

Challenge 1: Software Application Domains

- **Engineering/scientific software** – programs serving specific scientific fields like astronomy, computer aided design

Challenge 2: Legacy Software

- **What is a Legacy Software?**
- Legacy software systems **were developed decades ago** and have been continually modified to meet changes in business requirements and computing platforms.
- Such systems cause headaches for large organizations who find them costly to maintain and risky to evolve.
- How about we simply throw them away and build other ones?

Challenge 3: The Changing Nature of Software

- Four broad categories of software are evolving to dominate the industry. These categories were in their infancy little more than a decade ago:
  - Web applications
  - Mobile applications
  - Cloud computing
  - Software Product Lines
- Such changing nature demands the software to be flexible enough to cope with them.

Outline

- Is Software such a Challenging Thing?
- **What is Software Engineering?**
- Software Project Failures
- Software Development Myths
- Steps to a Successful Software
- A Case Study
- Course Organization

Software Engineering: A Working Definition

Software Engineering is a collection of **techniques**, **methodologies** and **tools** that help with the production of

A high quality software system developed with a given *budget* before a given *deadline* while *change* occurs

Challenge: Dealing with complexity and change

Once I finished coding, I'm done. Right?

Requirements analysis

Design

Coding and debugging

Testing and verification

Maintenance

Software Engineering is more than Writing Code

As a software engineer, to build/maintain a software, you need to:

1. *Understand the problem* (requirements gathering and analysis).
2. *Plan a solution* (modeling and software design).
3. *Carry out the plan* (coding).
4. *Examine the result for accuracy* (testing and quality assurance)

Note where writing code fits here!

Outline

- Is Software such a Challenging Thing?
- What is Software Engineering?
- **Software Project Failures**
- Software Development Myths
- Steps to a Successful Software
- A Case Study
- Course Organization

Software Project Failures

Kindergarten invitation!

- In 1992, Mary from Winona, Minnesota received an invitation to attend a kindergarten.
  - Mary was 104 years old at that time.

Prisoners released!

- In 2015, it was discovered that over 3,200 US prisoners have been released early because of an updated calculation.
  - Problem has been ongoing for 13 years
  - One prisoner was released 600 days early

Software Failures

Sydney's Water Corporation

- Attempted to introduce an automated customer information and billing system.
- The project had **inadequate planning and specifications**, leading to various changes and accordingly added costs/delays
- After spending US $33.2 million, the project was aborted

Software Project Failures

London Stock Exchange

- Decided to automate its system for settling stock transactions.
- After seven years of **overly complex design**, and poor project management, the project got cancelled.
- Cost: $600 million

Software Project Failures

Therac-25

- A computer controlled radiation therapy with **two modes of operation: one low beam mode for mild infections**, and **one high beam mode (through a filter) for serious infections**
- Due to design and coding errors that were not captured within design or code review **(testing)**, 6 persons died due to the treatment.

Outline

- Is Software such a Challenging Thing?
- What is Software Engineering?
- Software Project Failures
- **Software Development Myths**
- Steps to a Successful Software
- A Case Study
- Course Organization

Software Development Myths

Management Myths

- **Myth 1:** *We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know?*
- Reality: **The book of standards may very well exist, but is it used? Are software practitioners aware of its existence?**

Software Development Myths

Management Myths

- **Myth 2:** *If we get behind schedule, we can add more programmers and catch up.*
- **Reality**: ***"adding people to a late software project makes it later."*** **As new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort.**
- **People can be added but only in a planned and well-coordinated manner.**

Software Development Myths

Management Myths

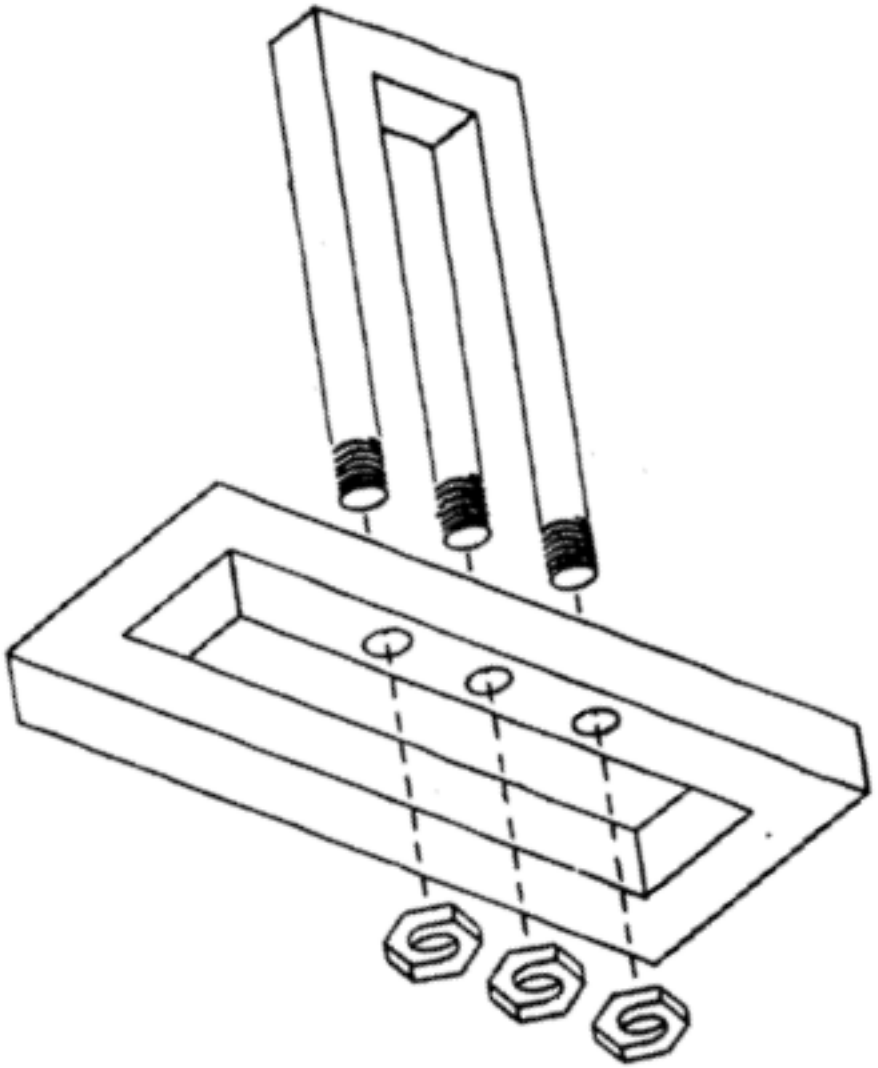
- **Myth 3:** If I decide to outsource the software project to a third party, I can just relax and let that firm build it.
- **Reality: If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.**

Software Development Myths

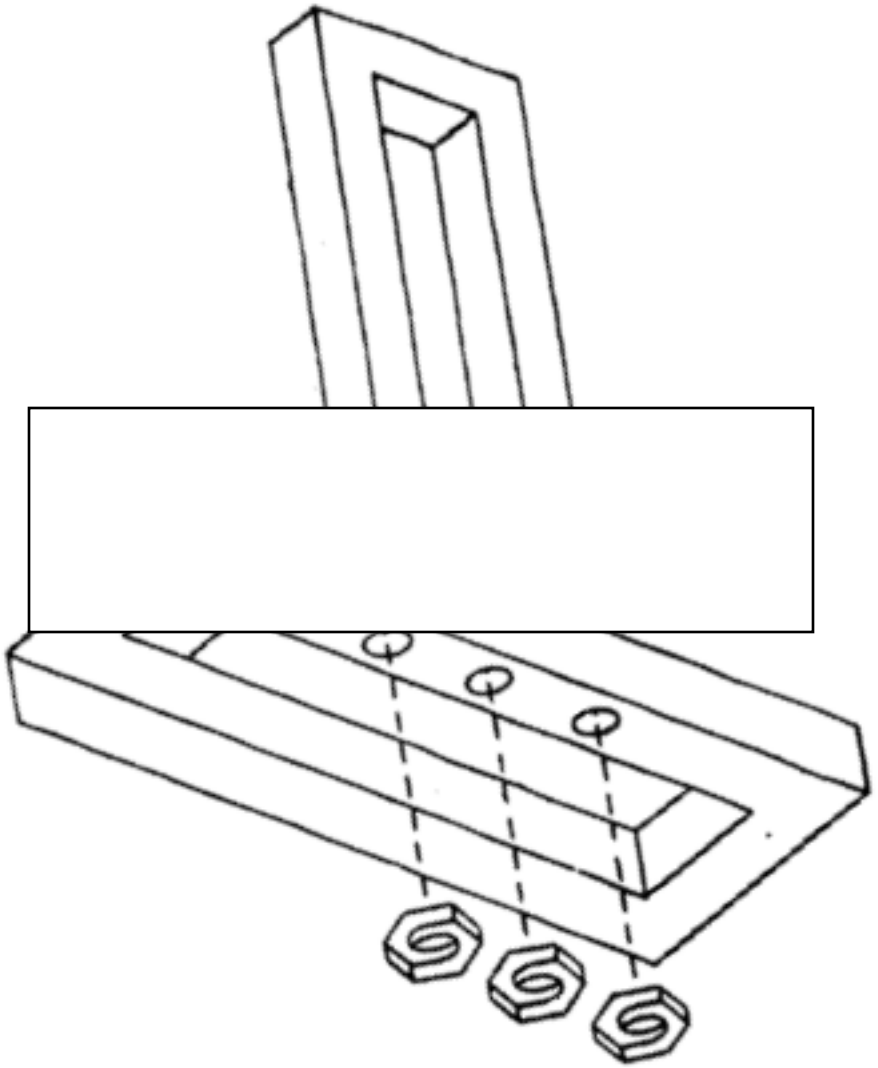
Customer Myths

- **Myth 1:** A general statement of objectives is sufficient to begin writing programs—we can fill in the details later.
- **Reality: An ambiguous "statement of objectives" is a recipe for disaster.**
- **Unambiguous requirements are developed only through effective and continuous communication between customer and developer.**
- **But…What is an ambiguous requirement?**

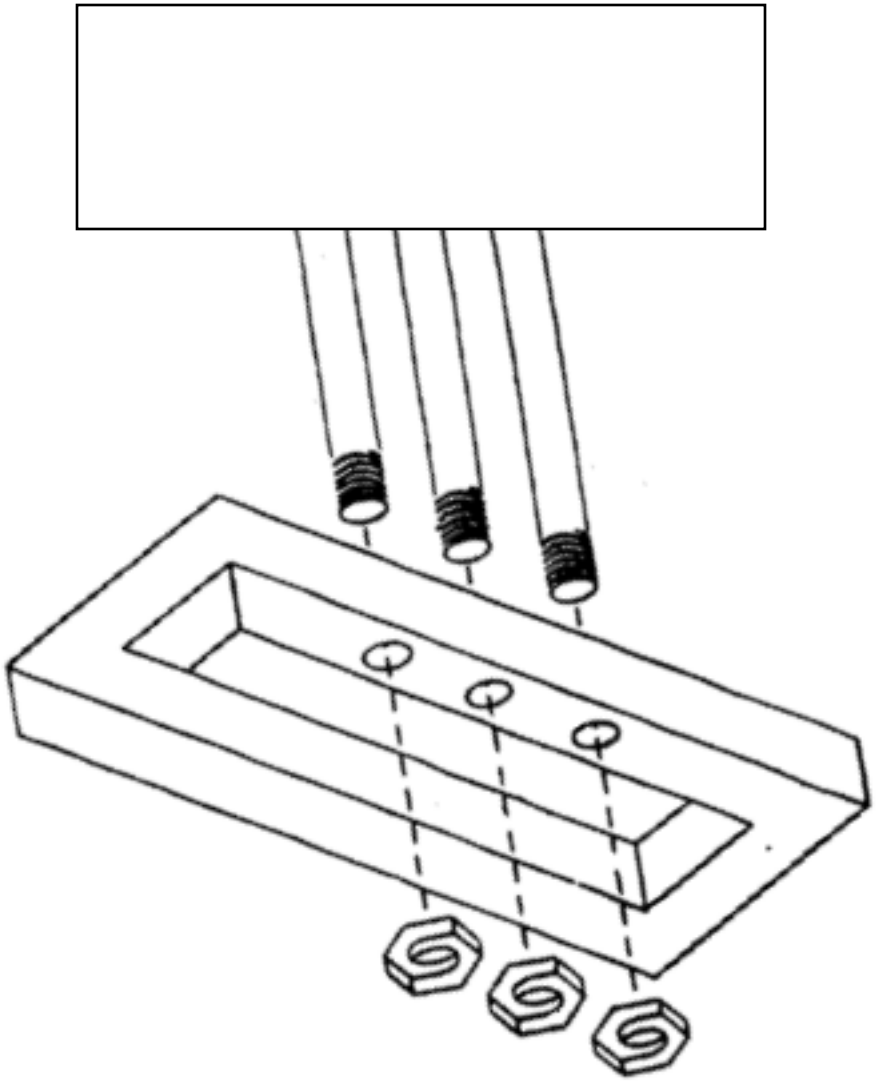
Can you develop this system?



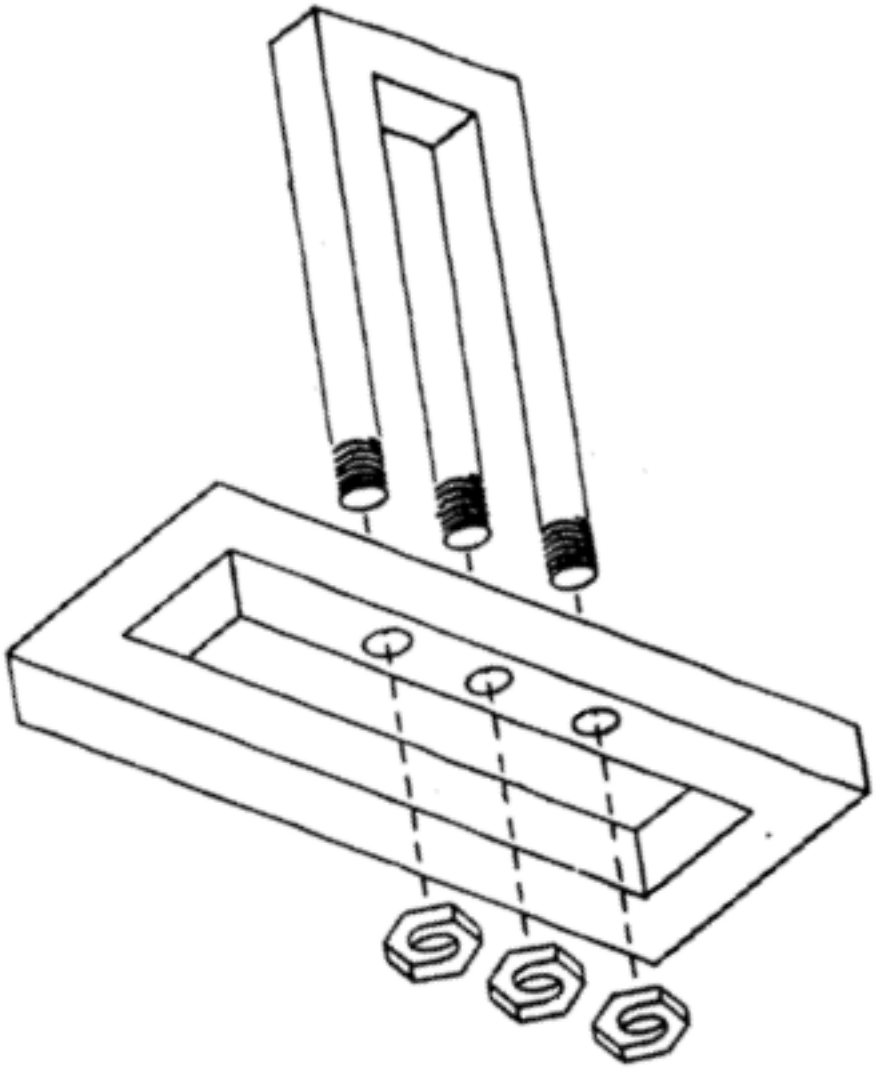
Can you develop this system?



Can you develop this system?



Can you develop this system?



The impossible Fork

Physical Model of the impossible Fork (Shigeo Fukuda)

Software Development Myths

Customer Myths

- **Myth 2:** Software requirements continually change, but change can be easily accommodated because software is flexible.
- **Reality: It is true that software requirements change, but the impact of change varies with the time at which it is introduced.**
- **When requirements changes are requested early (before design or code. has been started), the cost impact is relatively small.**
- **However, as time passes, the cost impact grows rapidly**

Outline

- Is Software such a Challenging Thing?
- What is Software Engineering?
- Software Project Failures
- Software Development Myths
- **Steps to a Successful Software**
- A Case Study
- Course Organization

Steps to Great Software

How do you write great software, every time?

Steps to Great Software

1. Make sure your software does what the customer wants it to do.

2. Apply basic OO principles to add flexibility.

3. Strive for a maintainable, reusable design.

What does maintainable mean?

What is the one constant in Software?

- No matter how well you design an application, over time the application will always grow and change.
- Your good analysis and design should make such changes an easy/doable task!
- **But... Do applications always change/grow?**

Outline

- Is Software such a Challenging Thing?
- What is Software Engineering?
- Software Project Failures
- Software Development Myths
- Steps to a Successful Software
- A Case Study
- **Course Organization**

Course Learning Objectives

- Appreciate the importance of Software Engineering
- Understand the big picture of the software development life cycle
- Apply requirements gathering and requirements elicitation techniques to properly write a requirements document for a software system
- Apply object-oriented principles and design principles to build a software design that could easily be maintained and extended to endure new requirements
- Apply basic design patterns to enhance your design
- Apply different testing techniques to adequately test some software system.

(Tentative) Evaluation

- Midterm (20 marks)
- Assignments/Quizzes/Lab Exam (20 marks)
- Final exam (60 marks)

Evaluation (Cont'd)

- Cheating Policy
  - There will be ZERO tolerance for any sort of cheating.
  - COPYING your code from online resources IS CHEATING
  - You are expected to submit your OWN ORIGINAL work for the graded course work.
  - Discussing the details of your solution with your colleague is CHEATING
  - When in doubt, then it is probably cheating!

Course Material

- Textbooks
  - McLaughlin, Brett, Gary Pollice, and David West. Head First Object-Oriented Analysis and Design: A Brain Friendly Guide to OOA&D. " O'Reilly Media, Inc.", 2007.
  - Bernd Bruegge, Allen H. Dutoit, "***Object-Oriented Software Engineering: Using UML, Patterns and Java***", 3rd Edition, Prentice Hall, Upper Saddle River, NJ, 2009;
- Additional readings may be added during each lecture.